

TiNPDW - Projekt

Porównanie podejść: PySpark RDD, PySpark SQL, PostgreSQL

Krzysztof Nasuta, nr indeksu 193328

1. Zbiór danych

W projekcie wykorzystano zbiór NYPD Complaint Data Historic, czyli historyczne zgłoszenia przestępstw w Nowym Jorku. Plik `data.csv` ma rozmiar ok. 3,01 GB i zawiera ponad 9,4 mln rekordów.

W analizie wykorzystano następujące kolumny:

- `OFNS_DESC` — opis rodzaju przestępstwa lub wykroczenia,
- `BORO_NM` — dzielnica Nowego Jorku, w której zarejestrowano zdarzenie,
- `JURIS_DESC` — jednostka lub jurysdykcja obsługująca zgłoszenie,
- `PREM_TYP_DESC` — typ miejsca zdarzenia,
- `VIC_AGE_GROUP` — grupa wiekowa ofiary.

2. Środowisko i implementacja

Projekt zaimplementowano w Pythonie. Trzy podejścia uruchamiane są przez główny skrypt projektu, wyniki i czasy zapisywane są do pliku JSON, a raport zbiorczy do pliku Markdown. PostgreSQL uruchamiany jest w kontenerze Docker.

Pomiary wykonano na komputerze z następującym środowiskiem:

Składnik	Wersja
CPU	AMD Ryzen 5 3600, sześciordzeniowy
System	Linux 6.19
Python	3.14.3
PySpark	4.1.1
PostgreSQL	18.3

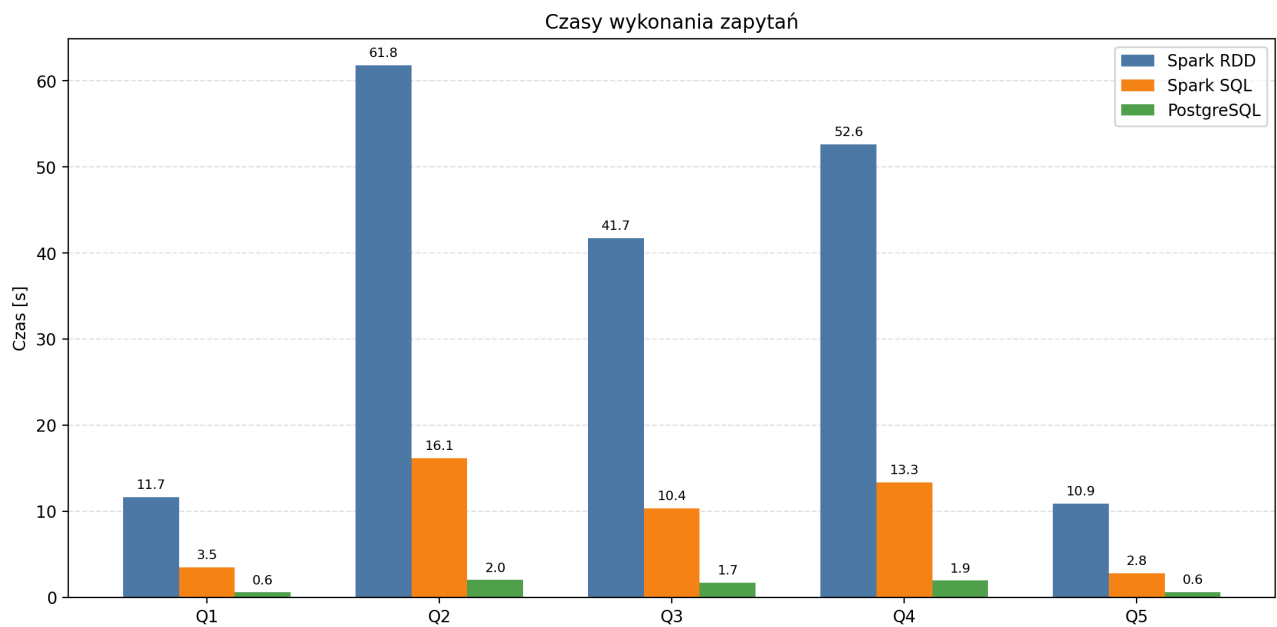
Podejście	Opis implementacji
PySpark RDD (bez <code>pyspark.sql</code>)	CSV wczytywany jako tekst (<code>SparkContext.textFile</code>), potem ręczne parsowanie linii i agregacje na RDD (<code>map</code> , <code>filter</code> , <code>reduceByKey</code> , <code>groupByKey</code>).
PySpark SQL	CSV wczytywany jako <code>DataFrame</code> (<code>SparkSession.read.csv</code>), a zapytania wykonywane przez <code>spark.sql(...)</code> z automatyczną optymalizacją planu.
PostgreSQL	CSV wczytywany w pandasie po 200 tys. wierszy (<code>pandas.read_csv</code> z <code>chunksize</code>) i zapisywany do PostgreSQL metodą <code>DataFrame.to_sql</code> (przez <code>SQLAlchemy/psycopg2</code>). Po załadowaniu tworzone są indeksy B-tree: <code>idx_boro</code> , <code>idx_juris</code> , <code>idx_prem</code> , <code>idx_ofns</code> .

Pomiar czasu realizowany przez kontekstowy `Timer` (`time.perf_counter`). W przypadku podejścia 3 osobno mierzony jest czas wczytania CSV do bazy (`load_and_convert`) oraz czasy poszczególnych zapytań — sumaryczny czas przedstawiono w sekcji wniosków.

Q2 zaimplementowano iteracyjnie zgodnie z treścią zadania (lista dzielnic → pętla → top 3 dla każdej), dlatego w wariancie RDD zbiór jest skanowany od nowa dla każdej z 5 dzielnic — stąd czas Q2 (61,82 s) jest ok. 5 razy większy niż Q1 (11,65 s), mimo że oba zapytania operują na tej samej kolumnie OFNS_DESC.

3. Czasy wykonania

Etap	Spark RDD [s]	Spark SQL [s]	PostgreSQL [s]
Wczytanie / konwersja	11,74	2,85	179,28
Q1 — top 10 typów	11,65	3,48	0,56
Q2 — top 3 / dzielnica	61,82	16,14	2,00
Q3 — top 3 urzędy + top 3 skargi	41,71	10,35	1,71
Q4 — top 4 lokalizacje + top 3	52,63	13,33	1,93
Q5 — rozkład wiekowy ofiar	10,86	2,80	0,58
Suma zapytań	178,67	46,10	6,78
Łącznie (wczyt. + zapytania)	190,41	48,95	186,06



Rysunek 1: Porównanie czasów wykonania zapytań Q1-Q5 dla trzech podejść.

4. Wyniki zapytań

Wyniki we wszystkich trzech podejściach są identyczne (po sortowaniu). Poniżej skrócone zestawienia.

4.1. Q1 — 10 najczęściej zgłaszanych skarg

OFNS_DESC	Liczba
PETIT LARCENY	1 666 745
HARRASSMENT 2	1 272 980
ASSAULT 3 & RELATED OFFENSES	998 322
CRIMINAL MISCHIEF & RELATED OF	916 268
GRAND LARCENY	831 964

DANGEROUS DRUGS	471 830
OFF. AGNST PUB ORD SENSBLY &	455 183
FELONY ASSAULT	393 369
ROBBERY	331 515
BURGLARY	310 292

4.2. Q2 — 3 najczęstsze skargi w każdej dzielnicy

Borough	OFNS_DESC	Liczba
BRONX	HARRASSMENT 2	281 854
BRONX	PETIT LARCENY	281 235
BRONX	ASSAULT 3 & RELATED OFFENSES	249 810
QUEENS	PETIT LARCENY	336 803
QUEENS	HARRASSMENT 2	270 699
QUEENS	CRIMINAL MISCHIEF & RELATED OF	211 190
MANHATTAN	PETIT LARCENY	523 016
MANHATTAN	GRAND LARCENY	312 754
MANHATTAN	HARRASSMENT 2	255 295
BROOKLYN	PETIT LARCENY	453 861
BROOKLYN	HARRASSMENT 2	380 140
BROOKLYN	ASSAULT 3 & RELATED OFFENSES	301 087
STATEN ISLAND	HARRASSMENT 2	84 381
STATEN ISLAND	PETIT LARCENY	67 656
STATEN ISLAND	CRIMINAL MISCHIEF & RELATED OF	56 801

4.3. Q3 — 3 najczęstsze urzędy i ich top 3 skargi

JURIS_DESC	OFNS_DESC	Liczba
N.Y. POLICE DEPT	PETIT LARCENY	1 595 343
N.Y. POLICE DEPT	HARRASSMENT 2	1 128 620
N.Y. POLICE DEPT	ASSAULT 3 & RELATED OFFENSES	868 949
N.Y. HOUSING POLICE	HARRASSMENT 2	124 950
N.Y. HOUSING POLICE	ASSAULT 3 & RELATED OFFENSES	100 417
N.Y. HOUSING POLICE	DANGEROUS DRUGS	78 186
N.Y. TRANSIT POLICE	GRAND LARCENY	26 541
N.Y. TRANSIT POLICE	CRIMINAL MISCHIEF & RELATED OF	24 574
N.Y. TRANSIT POLICE	ASSAULT 3 & RELATED OFFENSES	20 532

4.4. Q4 — 4 najczęstsze lokalizacje i ich top 3 skargi

PREM_TYP_DESC	OFNS_DESC	Liczba
STREET	PETIT LARCENY	436 837
STREET	CRIMINAL MISCHIEF & RELATED OF	380 458
STREET	DANGEROUS DRUGS	280 424

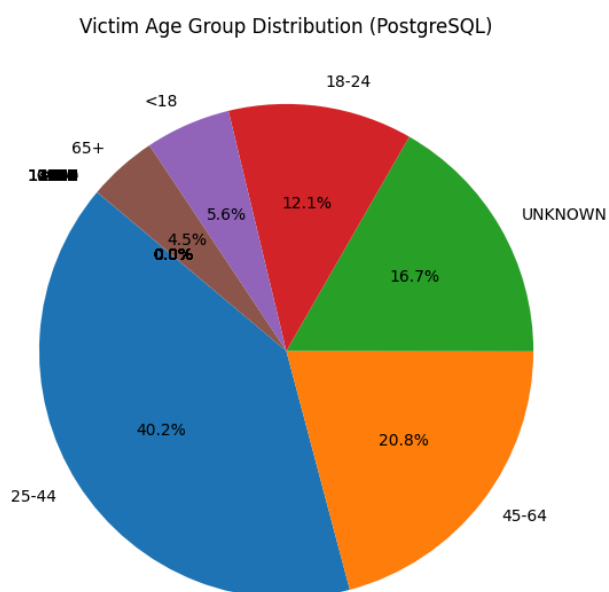
RESIDENCE - APT. HOUSE	HARRASSMENT 2	432 687
RESIDENCE - APT. HOUSE	ASSAULT 3 & RELATED OFFENSES	299 544
RESIDENCE - APT. HOUSE	OFF. AGNST PUB ORD SENSBLTY &	197 749
RESIDENCE-HOUSE	HARRASSMENT 2	196 184
RESIDENCE-HOUSE	ASSAULT 3 & RELATED OFFENSES	108 307
RESIDENCE-HOUSE	OFF. AGNST PUB ORD SENSBLTY &	97 755
RESIDENCE - PUBLIC HOUSING	HARRASSMENT 2	124 410
RESIDENCE - PUBLIC HOUSING	ASSAULT 3 & RELATED OFFENSES	99 835
RESIDENCE - PUBLIC HOUSING	DANGEROUS DRUGS	76 737

4.5. Q5 — rozkład wiekowy ofiar (VIC_AGE_GROUP)

Pominięto setki wartości nieprawidłowych (np. 930, -961, 11210), których łączny udział jest pomijalnie mały (poniżej 0,01%). W tabeli przedstawiono kategorie prawidłowe oraz UNKNOWN / (null).

Grupa wiekowa	Liczba	Udział
25 - 44	3 166 819	33,5%
45 - 64	1 639 842	17,3%
(null)	1 623 568	17,2%
UNKNOWN	1 316 479	13,9%
18 - 24	948 535	10,0%
<18	439 570	4,6%
65+	356 382	3,8%

Wykresy kołowe dla wszystkich trzech podejść wyglądają identycznie.



Rysunek 2: Rozkład grup wiekowych ofiar — Q5 (PostgreSQL).

5. Wnioski

1. Poprawność. Wszystkie trzy podejścia zwracają identyczne wyniki dla każdego z pięciu zapytań — daje to wzajemną walidację implementacji.
2. Najszybsze zapytania wykonuje PostgreSQL. Po wczytaniu danych do bazy i zbudowaniu indeksów (na `BORO_NM`, `JURIS_DESC`, `PREM_TYP_DESC`, `OFNS_DESC`) wszystkie zapytania wykonują się w ułamkach sekundy (łącznie 6,78 s). To efekt wykorzystania indeksów oraz optymalizatora zapytań PostgreSQL.
3. Najwolniejsze przetwarzanie zapytań ma Spark RDD. W przeciwieństwie do Spark SQL, RDD nie korzysta z Catalyst, czyli optymalizatora planu zapytań, i każdorazowo re-parsuje linie CSV, przez co jest ok. 26 razy wolniejszy od PostgreSQL na samych zapytaniach (178,67 s vs 6,78 s) oraz ok. 3,9 razy wolniejszy od Spark SQL. Dodatkowo zapytania Q2 - Q4 iterują po dzielnicach/urzędach/lokalizacjach, za każdym razem skanując RDD od nowa — koszt rośnie liniowo.
4. Spark SQL jest dobrym kompromisem. Catalyst i kolumnowe wnioskowanie typów redukuje czas zapytań do 46,10 s (ok. 3,9 razy szybciej niż RDD). Najlepsza opcja, gdy nie chcemy ponosić kosztu importu do RDBMS.
5. Charakter podejść. Spark lepiej sprawdza się w analizie ad hoc bez etapu trwałego ładowania danych, natomiast PostgreSQL daje największe korzyści wtedy, gdy dane są już załadowane i zapytania wykonywane są wielokrotnie.
6. Koszt importu w PostgreSQL dominuje. `load_and_convert` zajmuje 179,28 s, czyli ponad 26 razy więcej niż wszystkie zapytania razem. W rezultacie łączny czas (import + zapytania = 186,06 s) jest niemal taki sam jak dla Spark RDD (190,41 s) i ok. 3,8 razy gorszy niż Spark SQL (48,95 s).
7. Wniosek praktyczny. Dla jednorazowej analizy 3 GB CSV najszybszym rozwiązaniem jest Spark SQL. Gdy zapytania mają być wykonywane wielokrotnie, opłaca się jednorazowo zaimportować dane do PostgreSQL i korzystać z indeksów — czasy odpowiedzi spadają wtedy do poziomu sekund.